Лабораторная работа №7

Дискретное логарифмирование в конечном поле

Кадрова Ангелина Александровна

Содержание

1 Введение **4**
1.1 Цели и задачи 4
1.2 Теоретическое введение 4

2 Выполнение лабораторной работы **6**
2.1 Алгоритм p-метода Полларда 6
2.2 Тестирование алгоритма 7

3 Вывод **9**

Список литературы **10**

Список иллюстраций

2.1 Результаты работы алгоритма 8

1 Введение

1.1 Цели и задачи

Цель работы

Основная цель работы — изучить и реализовать алгоритм Полларда для дискретного логарифмирования в конечном поле.

Задание

С помощью языка программирования Julia реализовать алгоритм Полларда для дискретного логарифмирования в конечном поле.

1.2 Теоретическое введение

Задача дискретного логарифмирования применяется во многих алгоритмах криптографии с открытым ключом.

Обозначим $F_p = \mathbb{Z}/p\mathbb{Z}$, где p — простое целое число, и назовем его конечным полем из p элементов. Задача дискретного логарифмирования в конечном поле формулируется так: для данных целых чисел a и b ($a > 1$, $b < p$) найти логарифм — такое целое число x, что $a^x \equiv b \pmod{p}$.

Безопасность соответствующих криптосистем основана на том, что, зная числа a, x, p, вычислить $a^x \pmod{p}$ легко, а решить задачу дискретного логарифмирования трудно.

Для решения задачи применяется ρ-метод Полларда. При этом случайное отображение f должно обладать сжимающими свойствами и вычислимостью

логарифма. В качестве случайного отображения f обычно используют ветвящиеся отображения, например:

$$f(c) = \begin{cases} a \cdot c, & \text{при } c < p/2 \\ b \cdot c, & \text{при } c > p/2 \end{cases}$$

При $c < p/2$ имеем $\log_a f(c) = \log_a c + 1$, при $c > p/2 - \log_a f(c) = \log_a c + x$.

Алгоритм, реализующий p-Метод Полларда для задач дискретного логарифмирования:

Вход. Простое число p, число a порядка r по модулю p, целое число b, $1 < b < p$; отображение f, обладающее сжимающими свойствами и сохраняющее вычислимость логарифма.

Выход. Показатель x, для которого $a^x \equiv b \pmod{p}$, если такой показатель существует. 1. Выбрать произвольные целые числа u, v и положить $c = a^u \cdot b^v \pmod{p}, d = c$. 2. Выполнять $c = f(c) \pmod{p}, d = f(f(d)) \pmod{p}$, вычисляя при этом логарифмы для c и d как линейные функции от x по модулю r, до получения равенства $c = d \pmod{p}$. 3. Приравняв логарифмы для c и d, вычислить логарифм x решением сравнения по модулю r. Результат: x или “Решения нет”[1].

Julia — высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений[2]-[3].

2 Выполнение лабораторной работы

2.1 Алгоритм p-метода Полларда

Реализуем алгоритм Полларда для дискретного логарифмирования в конечном поле.

```
function next_step(val, u, v, a, b, p, r)
    if val < div(p, 2)
        return (a * val) % p, (u + 1) % r, v
    else
        return (b * val) % p, u, (v + 1) % r
    end
end

function pollard_rho(a, b, p, r)
    u = 2
    v = 2
    c = (powermod(a, u, p) * powermod(b, v, p)) % p
    d = c
    uc, vc = u, v
    ud, vd = u, v
    while true
        c, uc, vc = next_step(c, uc, vc, a, b, p, r)
        d, ud, vd = next_step(d, ud, vd, a, b, p, r)
```

```
        d, ud, vd = next_step(d, ud, vd, a, b, p, r)
        if c == d
            break
        end
    end
    A = (vc - vd) % r
    B = (ud - uc) % r
    if A == 0
        if B == 0
            return "Тривиальное совпадение"
        else
            return "Решения нет"
        end
    end
    x = (B * invmod(A, r)) % r
    return x
end
```

2.2 Тестирование алгоритма

```
a = 10
b = 64
p = 107
r = 53
println("Решаем задачу: $a^x = $b (mod $p)")
x = pollard_rho(a, b, p, r)
println("Найден логарифм x = $x")
println("Проверка: powermod($a, $x, $p) = ", powermod(a, x, p))
```

В результате получим следующий вывод(рис. 2.1):

```julia
a = 10
b = 64
p = 107
r = 53
println("Решаем задачу: $a^x ≡ $b (mod $p)")
x = pollard_rho(a, b, p, r)
println("Найден логарифм x = $x")
println("Проверка: powermod($a, $x, $p) ≡ ", powermod(a, x, p))
```

✓ 0.0s Julia

```
Решаем задачу: 10^x ≡ 64 (mod 107)
Найден логарифм x = 20
Проверка: powermod(10, 20, 107) ≡ 64
```

Рис. 2.1: Результаты работы алгоритма

3 Вывод

С помощью языка программирования Julia был реализован алгоритм Полларда для дискретного логарифмирования в конечном поле.

Список литературы

1. Математические основы защиты информации и информационной безопасности. Лабораторная работа №7: Дискретное логарифмирование в конечном поле: Методические указания к лабораторной работе. Саратовский государственный университет имени Н.Г. Чернышевского, 2026.
2. The Julia Programming Language [Электронный ресурс]. JuliaLang.org contributors, 2026. URL: https://julialang.org/ (дата обращения: 03.11.2026).
3. Julia Documentation [Электронный ресурс]. JuliaLang.org contributors, 2026. URL: https://docs.julialang.org/en/v1/ (дата обращения: 03.11.2026).